Reporte de Análisis: Métodos de Ordenamiento y Búsqueda

Materia: Análisis y Diseño de Algoritmos
****Mestro; MOrgan**

Reporte: Análisis y Rendimiento de Algoritmos

Este reporte analiza el rendimiento de seis métodos de ordenamiento y una búsqueda. El análisis se enfoca en cómo escala cada algoritmo utilizando un arreglo base de entrada.

Arreglo Base Fijo: ```python arr_base = [42, 15, 7, 38, 22, 50, 3, 19, 29, 11] target_search = 22

Copy

```
---

## Análisis y Código de Algoritmos

### 1. Bubble Sort (Ordenamiento de Burbuja)
> [!info] Análisis Teórico
> * **Complejidad:** $O(n^2)$.
> Compara elementos adyacentes y los intercambia. Es muy lento e ineficiente para listas grandes.

```python
```
```

```
def bubble_sort(arr):
    n = len(arr)
    index_of_last_unsorted = n
    while True:
        swapped = False
        for i in range(1, index_of_last_unsorted):
            if arr[i - 1] > arr[i]:
                arr[i - 1], arr[i] = arr[i], arr[i - 1]
                swapped = True
        index_of_last_unsorted -= 1
        if not swapped:
            break
    return arr
```

2. Selection Sort (Ordenamiento por Selección)

ⓘ Análisis Teórico

- **Complejidad: O(n2). Busca el elemento mínimo de la parte no ordenada y lo coloca al final de la parte ordenada.**

Python

Copy

```
def selection_sort(arr):
    num_of_elements = len(arr)
    for i in range(num_of_elements - 1):
        current_minimum = i
        for j in range(i + 1, num_of_elements):
            if arr[j] < arr[current_minimum]:
                current_minimum = j
        arr[i], arr[current_minimum] = arr[current_minimum], arr[i]
```

```
    return arr
```

3. Insertion Sort (Ordenamiento por Inserción)

ⓘ Análisis Teórico

- **Complejidad: O(n2). Extrae cada elemento y lo desplaza hacia la izquierda comparándolo con los elementos ordenados hasta encontrar su posición.**

Python

Copy

```python
def insertion_sort(arr):
    n = len(arr)
    for i in range(1, n):
        key = arr[i]
        j = i - 1
        while j ≥ 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr
```

4. Quick Sort

ⓘ Análisis Teórico

- **Complejidad (Promedio): O(nlogn). Utiliza la estrategia de Divide y Vencerás. Selecciona un pivote y particiona el arreglo evaluando si los**

demás elementos son menores.

Python

```python
import random

def quick_sort(arr, low=0, high=None):
    if high is None:
        high = len(arr) - 1
    if low < high:
        pivot = low
        store_index = pivot + 1
        for i in range(pivot + 1, high + 1):
            if arr[i] < arr[pivot] or (arr[i] == arr[pivot] and random.choice([True, False])):
                arr[i], arr[store_index] = arr[store_index], arr[i]
                store_index += 1
        arr[pivot], arr[store_index - 1] = arr[store_index - 1], arr[pivot]
        quick_sort(arr, low, store_index - 2)
        quick_sort(arr, store_index, high)
    return arr
```

5. Merge Sort

ⓘ Análisis Teórico

- **Complejidad: O(nlogn). Divide recursivamente el arreglo a la mitad y luego combina las particiones adyacentes ordenando los elementos.**

Python

```python
def merge_sort(arr):
    if len(arr) ≤ 1:
        return arr
    mid = len(arr) // 2
    left_part = merge_sort(arr[:mid])
    right_part = merge_sort(arr[mid:])
    return merge(left_part, right_part)

def merge(left_part, right_part):
    merged = []
    i = j = 0
    while i < len(left_part) and j < len(right_part):
        if left_part[i] ≤ right_part[j]:
            merged.append(left_part[i])
            i += 1
        else:
            merged.append(right_part[j])
            j += 1
    merged.extend(left_part[i:])
    merged.extend(right_part[j:])
    return merged
```

6. Counting Sort

ⓘ Análisis Teórico

- **Complejidad: O(n+k). No usa comparaciones directas. Cuenta las frecuencias de los elementos en un arreglo auxiliar y reconstruye la lista.**

Python

```python
def counting_sort(arr):
    if not arr:
        return arr
    max_val = max(arr)
    counting_array = [0] * (max_val + 1)
    for element in arr:
        counting_array[element] += 1
    list_idx = 0
    for key in range(len(counting_array)):
        while counting_array[key] > 0:
            arr[list_idx] = key
            list_idx += 1
            counting_array[key] -= 1
    return arr
```

7. Binary Search (Búsqueda Binaria)

⚠ **Aclaración Técnica Requiere estrictamente que los datos ya estén ordenados previamente.**

ⓘ **Análisis Teórico**

- **Complejidad: O(logn). Divide la lista ordenada a la mitad en cada paso, reduciendo drásticamente las comparaciones.**

Python

```python
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1
    while low ≤ high:
```

```
    mid = (high + low) //  2
    if arr[mid] < target:
      low = mid + 1
    elif arr[mid] > target:
      high = mid - 1
    else:
      return mid
  return -1
```

Conclusión Comparativa

La experimentación confirma exactamente la teoría asintótica (Big O) vista en clase:

1. **Los Cuadráticos (O(n2)): Bubble, Selection e Insertion Sort escalan muy mal con muchos datos. Hacen un exceso de intercambios y comparaciones. Experimentalmente, al duplicar el tamaño del arreglo, su tiempo se cuadruplica ($T(2n) \approx 4 \cdot T(n)$).**
2. **Divide y Vencerás (O(nlogn)): Quick Sort y Merge Sort son inmensamente superiores en la práctica. Al dividir el problema, las operaciones necesarias se reducen drásticamente, manteniendo tiempos casi planos frente a los cuadráticos al escalar la cantidad de datos.**
3. **Lineales y Búsqueda: Counting Sort (O(n+k)) es el más rápido al no hacer comparaciones, ideal para rangos pequeños. Por su parte, la Búsqueda Binaria (O(logn)) reduce las comparaciones hasta un 85% frente a una búsqueda normal, demostrando que ordenar los datos primero siempre vale la pena si se van a realizar múltiples consultas.**

En resumen: La elección de un algoritmo puede significar la diferencia entre horas y milisegundos. El mejor método siempre dependerá del volumen de los datos y la memoria disponible en el sistema.